arXiv:2303.08112v6 [cs.LG] 11 Nov 2025

Eliciting Latent Predictions from Transformers with the Tuned Lens

Nora Belrose [1 2] **Igor Ostrovsky** [1] **Lev McKinney** [3 2] **Zach Furman** [1 4] **Logan Smith** [1] **Danny Halawi** [1]
Stella Biderman [1] **Jacob Steinhardt** [5]

Abstract

We analyze transformers from the perspective of iterative inference, seeking to understand how model predictions are refined layer by layer. To do so, we train an affine probe for each block in a frozen pretrained model, making it possible to decode every hidden state into a distribution over the vocabulary. Our method, the *tuned lens*, is a refinement of the earlier "logit lens" technique, which yielded useful insights but is often brittle.

We test our method on various autoregressive language models with up to 20B parameters, showing it to be more predictive, reliable and unbiased than the logit lens. With causal experiments, we show the tuned lens uses similar features to the model itself. We also find the trajectory of latent predictions can be used to detect malicious inputs with high accuracy. All code needed to reproduce our results can be found at https://github.com/AlignmentResearch/tuned-lens.

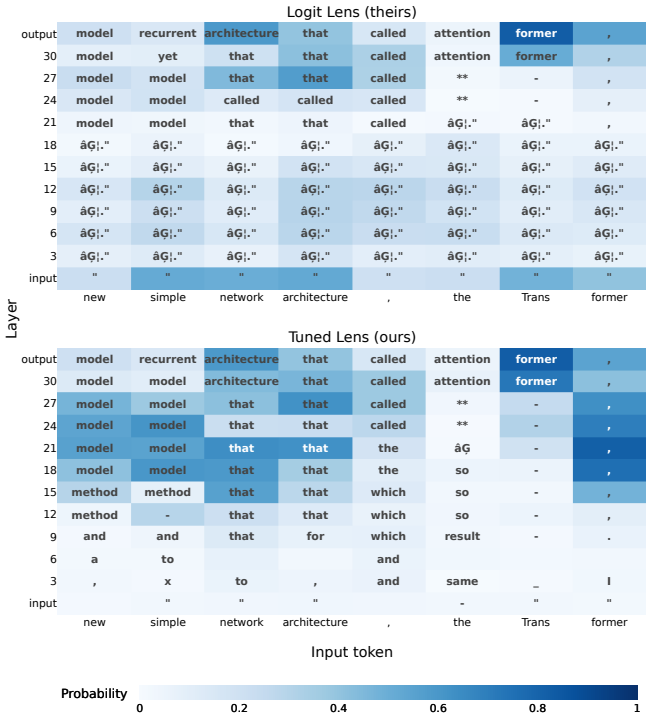

Figure 1. Comparison of our method, the *tuned lens* (bottom), with the "logit lens" (top) for GPT-Neo-2.7B prompted with an except from the abstract of Vaswani et al. (2017). Each cell shows the top-1 token predicted by the model at the given layer and token index. The logit lens fails to elicit interpretable predictions before layer 21, but our method succeeds.

1. Introduction

The impressive performance of transformers in natural language processing (Brown et al., 2020) and computer vision (Dosovitskiy et al., 2020) suggests that their internal representations have rich structure worthy of scientific investigation. One common approach is to train classifiers to extract specific concepts from hidden states, like part-of-speech and syntactic structure (Hewitt and Manning, 2019; Tucker et al., 2021; Li et al., 2022).

In this work, we instead examine transformer representations from the perspective of *iterative inference* (Jastrzębski et al., 2017). Specifically, we view each layer in a transformer language model as performing an incremental update to a latent prediction of the next token.[1] We decode these latent predictions through early exiting, converting the hidden state at each intermediate layer into a distribution over the vocabulary. This yields a sequence of distributions we call the *prediction trajectory*, which exhibits a strong tendency to converge smoothly to the final output distribution, with each successive layer achieving lower perplexity.

We build on the "logit lens" (nostalgebraist, 2020), an early exiting technique that directly decodes hidden states into vocabulary space using the model's pretrained unembedding matrix. We find the logit lens to be unreliable (Section 2), failing to elicit plausible predictions for models like BLOOM (Scao et al., 2022) and GPT Neo (Black et al., 2021). Even when the logit lens appears to work, its outputs are hard to interpret due to *representational drift*: features

[1]Eleuther AI [2]FAR AI [3]University of Toronto [4]Boston University [5]UC Berkeley. Correspondence to: Nora Belrose <nora@eleuther.ai>.

[1]See Appendix C for evidence supporting this view, including novel empirical results of our own.

may be represented differently at different layers of the network. Other early exiting procedures also exist (Schuster et al., 2022), but require modifying the training process, and so can't be used to analyze pretrained models.

To address the shortcomings of the logit lens, we introduce the *tuned lens*. We train L affine transformations, one for each layer of the network, with a distillation loss: transform the hidden state so that its image under the unembedding matches the final layer logits as closely as possible. We call these transformations *translators* because they "translate" representations from the basis used at one layer of the network to the basis expected at the final layer. Composing a translator with the pretrained unembedding yields a probe (Alain and Bengio, 2016) that maps a hidden state to a distribution over the vocabulary.

We find that tuned lens predictions have substantially lower perplexity than logit lens predictions, and are more representative of the final layer distribution. We also show that the features most influential on the tuned lens output are also influential on the model itself (Section 4). To do so, we introduce a novel algorithm called *causal basis extraction* (CBE) and use it to locate the directions in the residual stream with the highest influence on the tuned lens. We then ablate these directions in the corresponding model hidden states, and find that these features tend to be disproportionately influential on the model output.

We use the tuned lens to gain qualitative insight into the computational process of pretrained language models, by examining how their latent predictions evolve during a forward pass (Figure 1, Section B)

Finally, we apply the tuned lens in several ways: we extend the results of Halawi et al. (2023) to new models (Section 5.2), we find that tuned lens prediction trajectories can be used to detect prompt injection attacks (Perez and Ribeiro, 2022) often with near-perfect accuracy (Section 5.3), and find that data points which require many training steps to learn also tend to be classified in later layers (Section 5.4).

2. The Logit Lens

The logit lens was introduced by nostalgebraist (2020), who found that when the hidden states at each layer of GPT-2 (Radford et al., 2019) are decoded with the unembedding matrix, the resulting distributions converge roughly monotonically to the final answer. More recently it has been used by Halawi et al. (2023) to understand how transformers process few-shot demonstrations, and by Dar et al. (2022), Geva et al. (2022), and Millidge and Black (2022) to directly interpret transformer weight matrices.

The method. Consider a pre-LayerNorm transformer[2] $\mathcal{M}$.

[2]Both the logit lens and the tuned lens are designed primarily for the pre-LN architecture, which is more unambiguously iterative. Luckily pre-LN is by far more common than post-LN among state-of-the-art models. See Zhang and He (2020, Appendix C) for more discussion.

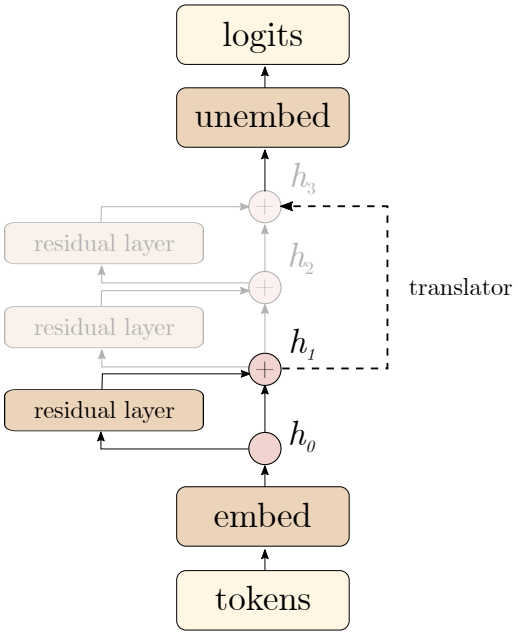

Figure 2. The tuned lens takes the hidden state at an intermediate layer (e.g. h_1 above), and applies a learned affine transformation (the translator). We then convert the hidden state into logits with the unembedding layer.

We'll decompose $\mathcal{M}$ into two "halves," $\mathcal{M}_{\leq \ell}$ and $\mathcal{M}_{>\ell}$. The function $\mathcal{M}_{\leq \ell}$ consists of the layers of $\mathcal{M}$ up to and including layer ℓ, and it maps the input space to hidden states. Conversely, the function $\mathcal{M}_{>\ell}$ consists of the layers of $\mathcal{M}$ after ℓ, which map hidden states to logits.

The transformer layer at index ℓ updates the representation as follows:

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + F_\ell(\boldsymbol{h}_\ell), \tag{1}$$

where F_ℓ is the residual output of layer ℓ. Applying Equation 1 recursively, the output logits $\mathcal{M}_{>\ell}$ can be written as a function of an arbitrary hidden state $\boldsymbol{h}_\ell$ at layer ℓ:

$$\mathcal{M}_{>\ell}(\boldsymbol{h}_\ell) = \text{LayerNorm}\Big[\boldsymbol{h}_\ell + \sum_{\ell'=\ell}^{L} \underbrace{F_{\ell'}(\boldsymbol{h}_{\ell'})}_{\text{residual update}}\Big] W_U. \tag{2}$$

The logit lens consists of setting the residuals to zero:

$$\text{LogitLens}(\boldsymbol{h}_\ell) = \text{LayerNorm}[\boldsymbol{h}_\ell] W_U. \tag{3}$$

While this simple form of the logit lens works reasonably well for GPT-2, nostalgebraist (2021) found that it fails to extract useful information from the GPT-Neo family of models (Black et al., 2021). In an attempt to make the method

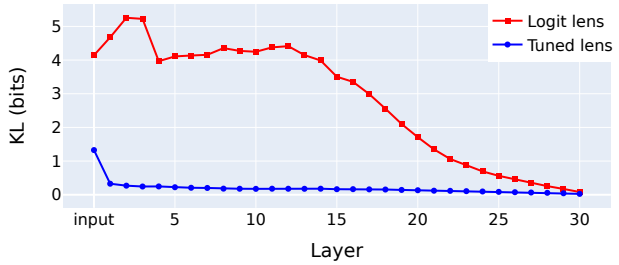

Figure 3. Bias of logit lens and tuned lens outputs relative to the final layer output for GPT-Neo-2.7B. The last transformer layer is included for both probes. Unlike the tuned lens, the logit lens is systematically biased toward some vocabulary items over others until the very end of the network.

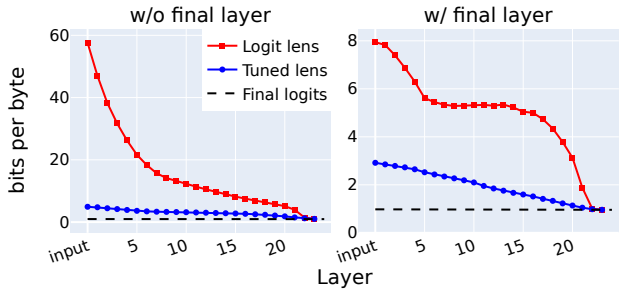

Figure 4. Perplexity of predictions elicited from BLOOM 560M under four conditions: the logit lens (red squares) and the tuned lens (blue circles), and including (left) and excluding (right) the final transformer layer from the probe. We find that tuned lens predictions have substantially lower perplexity whether or not the final layer is included, showing it is an independent and complementary proposal.

work better for GPT-Neo, they introduce an extension which retains the last transformer layer, yielding:

$$\mathrm{LogitLens}^{\mathrm{ext}}(\boldsymbol{h}_\ell) = \mathrm{LayerNorm}[\boldsymbol{h}_\ell + F_L(\boldsymbol{h}_\ell)]W_U \quad (4)$$

This extension is only partially successful at recovering meaningful results; see Figure 1 (top) for an example.

Unreliability. Beyond GPT-Neo, the logit lens struggles to elicit predictions from several other models released since its introduction, such as BLOOM (Scao et al., 2022) and OPT 125M (Zhang et al., 2022) (Figure 14).

Moreover, the type of information extracted by the logit lens varies both from model to model and from layer to layer, making it difficult to interpret. For example, we find that for BLOOM and OPT 125M, the top 1 prediction of the logit lens is often the *input* token, rather than any plausible continuation token, in more than half the layers (Figure 18).

Bias. Even when the logit lens is useful, we find that it is a *biased* estimator of the model's final output: it systematically puts more probability mass on certain vocabulary items than the final layer does.

This is concerning because it suggests we can't interpret the logit lens prediction trajectory as a belief updating in response to new evidence. The beliefs of a rational agent should not update in an easily predictable direction over time (Yudkowsky, 2007), since predictable updates can be exploited via Dutch books (Ramsey, 1926; De Finetti et al., 1937; Hacking, 1967; Garrabrant et al., 2016). Biased logit lens outputs are trivially exploitable once the direction of bias is known: one could simply "bet" against the logit lens at layer $\ell < L$ that the next token will be one of the tokens that it systematically downweights, and make unbounded profit in expectation.

Let $\boldsymbol{x}$ be a sequence of tokens sampled from a dataset D, and let $\boldsymbol{x}_{<t}$ refer to the tokens preceding position t in the sequence. Let $q_\ell(\cdot|\boldsymbol{x}_{<t})$ be the logit lens distribution at layer ℓ for position t, and let $p(\cdot|\boldsymbol{x}_{<t})$ be the final layer distribution for position t.

We define $p(v|\boldsymbol{x})$ to be the probability assigned to a vocabulary item v in a sequence $\boldsymbol{x}$, averaged over all positions $1 \ldots T$:

$$p(v|\boldsymbol{x}) \stackrel{\text{def}}{=} \frac{1}{T}\sum_{t=1}^{T} p(v|\boldsymbol{x}_{<t}). \quad (5)$$

Slightly abusing terminology, we say that q_ℓ is an "unbiased estimator" of p if, for every item v in the vocabulary, the probability assigned to v averaged across all tokens in the dataset is the same:

$$\begin{gathered} \mathop{\mathbb{E}}_{\boldsymbol{x}\in D}\Big[q_\ell(v|\boldsymbol{x})\Big] = \mathop{\mathbb{E}}_{\boldsymbol{x}\in D}\Big[p(v|\boldsymbol{x})\Big] \\ q_\ell(v) = p(v) \\ \forall v \in \mathcal{V}, \quad \mathcal{V} = \{\texttt{"aardvark"}, \ldots\} \end{gathered} \quad (6)$$

In practice, Equation 6 will never hold exactly. We measure the degree of bias using the KL divergence between the marginal distributions, $D_{KL}(p \,||\, q_\ell)$.

In Figure 3 we evaluate the bias for each layer of GPT-Neo-2.7B. We find the bias of the logit lens can be quite large: around 4 to 5 bits for most layers. As a point of comparison, the bias of Pythia 160M's final layer distribution relative to that of its larger cousin, Pythia 12B, is just 0.0068 bits.

3. The Tuned Lens

One problem with the logit lens is that, if transformer layers learn to output residuals that are far from zero *on average*, the input to LogitLens may be out-of-distribution and yield nonsensical results. In other words, the choice of zero as